



Universidad Autónoma de Nayarit

Unidad Académica De Economía

Sistemas Computacionales

“Semana 14 de 14 • Integración de Múltiples Estructuras”

Jorge Armando Navarrete Ruiz

Docente.- DR. Eligardo Cruz Sanchez

U. A. ESTRUCTURA DE BASES DE DATOS

Fase 1: COMPRENDE	3
Fase 2: APLICA	4
Fase 3: REFLEXIONA	5
Observación P95 vs Mediana:	5
# - lookup: P95 cercano a la mediana (bajo jitter).	5
# - rango/top10: P95 mayor por variación del SO/cache y por procesar más datos (k) y ordenar.	5
#	5
# Cuello de botella (n=10,000):	5
# - rango (Mediana 808.8930 us) es el más costoso -> domina el tiempo del sistema.	5
Fase 4: VALIDA	6

Fase 1: COMPRENDE

```
# COMPRENDE-1
# Inventario de Estructuras del Proyecto (SistemaMX)
#
# | Estructura | Datos que almacena |
# |-----|-----|
# | unordered_map<string, Pelicula> | Catálogo: película por ID |
# | BST (NodoBST por puntaje) | Índice secundario: películas agrupadas por puntaje |
# | vector<vector<Pelicula>> pilaResultados | Snapshots de resultados de consultas 'rango' |
# | vector<OperacionUndo> pilaUndo | Historial de inserciones (IDs insertados) |
```

Operación principal que resuelve	Complejidad esperada	Semana origen
lookup exacto por ID	$O(1)$ amortizado	S12
búsqueda por rango [minv,maxv] de puntajes	$O(h + k)$ (promedio $\sim O(\log n + k)$)	S13
reutilizar la última consulta para 'top10'	push_back $O(1)$, acceso back $O(1)$	S5 (pila)
deshacer última inserción (undo)	push_back $O(1)$, pop_back $O(1)$	S5 (pila)

COMPRENDE-2

Matriz de acoplamiento / Operaciones compuestas (mínimo 2)

#

Operación compuesta 1: rango(minv,maxv)

Paso 1: recorrer BST y recolectar resultados en rango

bst_rango(bst_raiz, minv, maxv, resultados) -> $O(\log n + k)$ promedio (BST), peor $O(n + k)$

Paso 2: guardar snapshot para consultas posteriores

pilaResultados.push_back(resultados) -> $O(1)$
(amortizado) + copia $O(k)$

Complejidad total: $O(\log n + k)$ (dominante: recorrido + k resultados)

#

Operación compuesta 2: top10() (depende del último rango)

Paso 1: obtener el último snapshot de resultados

base = pilaResultados.back() -> $O(1)$

Paso 2: ordenar resultados por puntaje desc / id asc

sort(base.begin(), base.end(), ...) -> $O(k \log k)$

Paso 3: recortar a 10 elementos

resize(10) -> $O(1)$

Complejidad total: $O(k \log k)$ (dominante: sort)

#

Operación compuesta 3 (extra útil): undo()

Paso 1: pop de pilaUndo -> $O(1)$

Paso 2: borrar del unordered_map -> O(1)
 amortizado
 # Paso 3: borrar del BST por (puntaje,id) -> O(log n)
 promedio / peor O(n)
 # Complejidad total: O(log n) promedio (dominante: borrar en
 BST)

Fase 2: APLICA

APLICA-3

Clase SistemaMX — Inicialización
 # Estructuras inicializadas:
 # 1) catalogo: unordered_map<string, Pelicula> para lookup
 O(1)
 # 2) bst_raiz: puntero a BST (NodoBST) para consultas por
 rango
 # 3) pilaResultados: pila (vector) para snapshots de consultas
 rango
 # 4) pilaUndo: pila (vector) para deshacer inserciones
 #
 # Verificación observada en ejecución:
 # catalogo size: 0
 # bst vacio: 1
 # pilaResultados vacia: 0
 # pilaUndo vacia: 0

APLICA-4

Método agregar_contenido(id,titulo,puntaje) — Inserción
 consistente
 # Valida:
 # - ID duplicado -> invalid_argument
 # - puntaje fuera de [0,10] -> out_of_range
 # Actualiza estructuras (invariante de integración):
 # - catalogo.insert({id, p}) -> Hash (O(1) amort.)
 # - bst_raiz = bst_insertar(bst_raiz, p) -> BST (O(log n)
 prom.)
 # - pilaUndo.push_back({id}) -> Pila undo (O(1))
 # Resultado esperado: después de cada inserción, el contenido es
 visible
 # desde lookup (catalogo) y desde rango (BST), y es reversible
 con undo.

APLICA-5

Método procesar_solicitud(tipo,...) — Enrutador / Fachada

Tipos implementados:

- "lookup" : busca por ID en unordered_map y devuelve vector con 0 o 1 elemento

- "rango" : valida $\text{minv} \leq \text{maxv}$, consulta BST, guarda snapshot en pilaResultados y devuelve resultados

- "top10" : ordena el último snapshot (requiere haber ejecutado "rango" antes) y recorta a 10

- "undo" : revierte la última inserción (borra en hash + borra en BST)

Tipo desconocido: lanza invalid_argument("Tipo desconocido")

Fase 3: REFLEXIONA

```
# REFLEXIONA-1
# Resultados reales de benchmark (microsegundos, us)
#
# | Operación | n=1,000 Mediana(us) | n=1,000 P95(us) | n=10,000 Mediana(us) |
# |-----|-----|-----|-----|
# | lookup   | 0.3645              | 0.5640           | 0.3610                |
# | rango    | 76.0165             | 104.1030          | 808.8930               |
# | top10    | 40.6105             | 59.4830           | 464.8990               |
#
# | n=10,000 P95(us) | Complejidad |
# |-----|-----|
# | 0.4050            | O(1) amort. |
# | 1082.2250         | O(log n + k) prom. (peor O(n+k)) |
# | 635.3440          | O(k log k) (ordena el snapshot del rango) |
```

Observación P95 vs Mediana:

- lookup: P95 cercano a la mediana (bajo jitter).

- rango/top10: P95 mayor por variación del SO/cache y por procesar más datos (k) y ordenar.

#

Cuello de botella (n=10,000):

- rango (Mediana 808.8930 us) es el más costoso -> domina el tiempo del sistema.

Fase 4: VALIDA

VALIDA (V-8)

- # Reporte de 5 casos: 2 dados + 3 edge cases del equipo
- #
- # Caso dado 1: lookup inexistente
- # - Entrada: ID no existente
- # - Esperado: resultado vacío (vector size 0)
- # - Real: OK (si no está en catalogo, no se agrega al vector)
- # - Estado:  PASA
- #
- # Caso dado 2: inserción consistente
- # - Acción: agregar_contenido("VTEST_99", ..., 5.5) y consultar por lookup y por rango
- # - Esperado: visible por lookup y aparece en rango [5.4,5.6]
- # - Real: OK (entra a unordered_map y BST)
- # - Estado:  PASA
- #
- # Edge case 1: rango inválido (minv > maxv)
- # - Entrada: procesar_solicitud("rango","", 9.0, 7.0)
- # - Esperado: lanzar invalid_argument("Rango invalido")
- # - Real: lanza invalid_argument
- # - Estado:  PASA
- #
- # Edge case 2: top10 sin ejecutar rango antes
- # - Entrada: procesar_solicitud("top10") con pilaResultados vacía
- # - Esperado: lanzar runtime_error("Ejecuta rango primero")
- # - Real: lanza runtime_error
- # - Estado:  PASA
- #
- # Edge case 3: undo sin nada que deshacer
- # - Entrada: procesar_solicitud("undo") con pilaUndo vacía
- # - Esperado: lanzar runtime_error("Nada que deshacer")
- # - Real: lanza runtime_error

- Estado:  PASA

VALIDA (V-9)

Invariante cruzada propuesta para este C++:

Inv-1: `catalogo.size() == cantidad_total_de_peliculas_en_BST`

Inv-2: `pilaUndo.size() == catalogo.size()`

Inv-3: BST en orden ascendente por clave (inorder)

#

Resultado esperado tras 50 inserciones aleatorias:

- Inv-1: 

- Inv-2: 

- Inv-3: 

#

Nota: Para Inv-1 en este diseño, cada `NodoBST` guarda `vector<Pelicula>`,

así que contar "nodos" no basta; se debe contar total de películas en todos los vectores.